Programming Fundamentals

Assignment 2

Group Members:

Hafiz Muhammad Asad **(022)**
Awais Zaheer **(036)**
Umer Zaheer **(026)**
Measum Raza **(032)**
Muhammad Fahad **(040)**

Program 1: Electricity Bill Management System for LESCO

Code:

```
#include <iostream>
#include <string>
using namespace std;

string customerName;
string customerId;
int metersInstalled;
char customerType;
int unitConsumed;
double fixedTotal = 0;
double cost = 0;
double dutyAmount = 0;
double taxAmount =0;
```

```
double slabRate[8] = {12.21, 14.53, 31.51, 38.41, 41.62, 43.04, 44.18,
49.10};
int fixedCharges[6] = {0,200,400,600,800,1000};
//Function Customer Registration
void registerCustomer()
{
	cout<<"Enter Your Name: ";
	cin.ignore();
	getline(cin,customerName);
	cout<<"Enter Your Customer ID: ";
	cin>>customerId;
	cout<<"Enter (H) for Household and (C) for Commercial
Connection: ";
	cin>>customerType;
	cout<<"Number of Meters Already Installed: ";
	cin>>metersInstalled ;
}
//Function Main Menu
void displayMenu()
{
	cout <<"\n=========== Main Menu ===========\n"<<endl;
		cout<<"1. Press 1 for Calculate Monthly Bill: "<<endl;
		cout<<"2. Press 2 Apply for New Connection: "<<endl;
		cout<<"3. Press 3 for View Customer Detail: "<<endl;
		cout<<"4. Press 4 for Exit Program: "<<endl;
}
```

```
//Function Customer Detail
void customerDetail()
{
	cout<<"Customer Name: "<<customerName <<endl;
	cout<<"Customer ID: "<<customerId <<endl;
	if (customerType == 'H' || customerType == 'h')
	{
		cout<<"Connection Type: Household Connection" <<endl;
	}
	else
	{
		cout<<"Connection Type: Commercial Connection"<<endl;
	}
	cout<<"Meters Installed: "<<metersInstalled <<endl;
}
//Function Total Monthly Bill
void monthlyBill()
{
   double totalBill = 0;
	cout<<"Enter Total Unit Consumed: ";
	cin>>unitConsumed;
	if (unitConsumed>=1 && unitConsumed<=100)
	{
		cost = slabRate[0] * unitConsumed;
	}
	else if (unitConsumed>=101 && unitConsumed<=200)
```

```
{
        cost = slabRate[1] * unitConsumed;
}
else if (unitConsumed>=201 && unitConsumed<=300)
{
        cost = slabRate[2] * unitConsumed;
}
else if (unitConsumed>=301 && unitConsumed<=400)
{
        cost = slabRate[3] * unitConsumed;
}
else if (unitConsumed>=401 && unitConsumed<=500)
{
        cost = slabRate[4] * unitConsumed;
}
else if (unitConsumed>=501 && unitConsumed<=600)
{
        cost = slabRate[5] * unitConsumed;
}
else if (unitConsumed>=601 && unitConsumed<=700)
{
        cost = slabRate[6] * unitConsumed;
}
else
{
        cost = slabRate[7] * unitConsumed;
```

```
        }
}
//      Apply Fixed Charges
void fixedCharge()
{
  if (unitConsumed>=1 && unitConsumed<=300)
  {
        fixedTotal = 0;
  }
  else if (unitConsumed>=301 && unitConsumed<=400)
  {
        fixedTotal = fixedCharges[1];
  }
  else if (unitConsumed>=401 && unitConsumed<=500)
  {
        fixedTotal = fixedCharges[2];
  }
  else if (unitConsumed>=501 && unitConsumed<=600)
  {
        fixedTotal = fixedCharges[3];
  }
  else if (unitConsumed>=601 && unitConsumed<=700)
  {
        fixedTotal = fixedCharges[4];
  }
  else
```

```
  {
        fixedTotal = fixedCharges[5];
  }
}
//Function Duty Charges
void dutyCharges()
{
        dutyAmount = cost * 0.015;
}
//Function Income Tax Calculate
void calculateIncome_Tax()
{
        if (customerType == 'H' || customerType == 'h')
        {
                taxAmount = cost * 0.10;
        }
        else
        {
                taxAmount = cost * 0.15;
        }
}
//Function Final Bill
void displayFinal_Bill()
{
        double meterRent = 250;
        double tvFee = 35;
```

```
        double subTotal;
        double gstAmount;
        double totalPayable;
        subTotal = cost + fixedTotal + dutyAmount + taxAmount +
meterRent + tvFee;
        gstAmount = subTotal * 0.18;
        totalPayable = gstAmount + subTotal;
        cout<<"\n========== LESCO ELECTRICITY BILL ==========\n"
<<endl;
        cout<<"Customer Name: "<<customerName <<endl;
        cout<<"Customer Id: " <<customerId <<endl;
        if (customerType == 'H' || customerType == 'h')
        {
                cout<<"Customer Type: Household Connection" <<endl;
        }
        else
        {
                cout<<"Customer Type: Commercial Connection"<<endl;
        }
        cout<<"Unit Consumed: " <<unitConsumed <<endl <<endl;
        cout<<"Electricity Consumption Charges: Rs."<<cost <<endl;
        cout<<"Electricity Duty: Rs." <<dutyAmount<<endl;
        cout<<"Fixed Charges: Rs." <<fixedTotal <<endl;
        cout<<"Meter Rent: Rs." <<meterRent <<endl;
        cout<<"TV Fee: Rs." <<tvFee <<endl;
        cout<<"GST Charges: Rs." <<gstAmount <<endl;
        cout<<"Income Tax: Rs." <<taxAmount <<endl <<endl;
```

```
        cout<<"Total Payable Bill: Rs." <<totalPayable <<endl;


}
//Function New Connection
void newConnection()
{
    double meterCost = 3500;
   double cableCost = 1500;
   double securityCost = 5000;
   double additionalCharges = 0;
   double properConnection_Cost = 0;
   double totalConnection_Charges;
   int choice;
   cout<<"Do you want a new proper connection? (Press 1 for Yes, 0 for
No): ";
   cin>>choice;
   if (choice == 1)
   {
        properConnection_Cost = 250000;
        }
        if (customerType == 'H' || customerType == 'h')
        {
                if (metersInstalled == 1 )
                {
                        additionalCharges = 2500;
                }
                else
```

```
            {
                additionalCharges = 5000;
            }
        }
        else
        {
            if (metersInstalled == 1)
            {
                additionalCharges = 35000;
            }
            else
            {
                additionalCharges = 70000;
            }
        }
        totalConnection_Charges = meterCost + cableCost + securityCost +
additionalCharges + properConnection_Cost;
        cout<<"Total New Connection Charges: Rs."
<<totalConnection_Charges <<endl;
}
int main()
{
        registerCustomer();
        int choice;
        do
        {
        displayMenu();
```

```
        cout<<"Enter Your Choice: ";
cin>>choice;

        if (choice == 1)
        {
monthlyBill();
fixedCharge();
dutyCharges();
calculateIncome_Tax();
displayFinal_Bill();

        }
        else if (choice == 2)
        {
                newConnection();
        }
        else if (choice == 3)
        {
                customerDetail();
        }
        else if (choice == 4)
        {
                cout<<"You Exit the Program";
        }
        else
        {
```

```
            cout<<"You Entered Invalid Choice";
        }
}
        while(choice!=4);
        return 0;
}
```

Program 2: Restaurant Billing Management System

Code:

```
#include <iostream>
#include <string>
using namespace std;

string customerName;
string contactNumber;
string orderType;

int persons;
double foodBill = 0;

string foodItems[8] = {
   "Chicken Burger",
   "Zinger Burger",
   "Pizza Small",
   "Pizza Large",
   "Chicken Biryani",
```

```
    "BBQ Platter",
    "Fries",
    "Cold Drink"
};
double prices[8] = {450, 550, 900, 1800, 350, 1200, 250, 120};
void registerCustomer()
{
    cout << "Enter Customer Name: ";
    cin.ignore();
    getline(cin, customerName);
    cout << "Enter Contact Number: ";
    getline(cin, contactNumber);
    cout << "Enter Order Type (Dine-in/Takeaway): ";
    getline(cin, orderType);
    cout << "Enter Number of Persons: ";
    cin >> persons;
}
void displayMenu()
{
    cout << "\n========== FOOD MENU ==========\n";
    for(int i = 0; i < 8; i++)
    {
        cout << i + 1 << ". " << foodItems[i]
             << " - Rs. " << prices[i] << endl;
    }
}
```

```
void placeOrder()
{
   int itemNo, quantity;
   char choice;
   do
   {
      displayMenu();
      cout << "\nEnter Item Number: ";
      cin >> itemNo;
      cout << "Enter Quantity: ";
      cin >> quantity;
      foodBill = foodBill + (prices[itemNo - 1] * quantity);
      cout << "Add More Items? (y/n): ";
      cin >> choice;
   } while(choice == 'y' || choice == 'Y');
}
double calculateServiceCharges()
{
   if(orderType == "Dine-in" || orderType == "dine-in")
   {
      return foodBill * 0.10;
   }
   else
   {
      return foodBill * 0.05;
   }
```

```
}
double calculateGST()
{
   return foodBill * 0.16;
}
double calculateDiscount()
{
   if(foodBill >= 3000 && foodBill <= 5000)
   {
      return foodBill * 0.05;
   }
   else if(foodBill > 5000 && foodBill <= 10000)
   {
      return foodBill * 0.10;
   }
   else if(foodBill > 10000)
   {
      return foodBill * 0.15;
   }
   else
   {
      return 0;
   }
}
void finalBill()
{
```

```
    double service = calculateServiceCharges();
    double gst = calculateGST();
    double discount = calculateDiscount();
    double total = foodBill + service + gst - discount;
    cout << "\n========== RESTAURANT BILL ==========\n";
    cout << "Customer Name: " << customerName << endl;
    cout << "Contact Number: " << contactNumber << endl;
    cout << "Order Type: " << orderType << endl;
    cout << "Number of Persons: " << persons << endl;
    cout << "\nFood Bill: Rs. " << foodBill << endl;
    cout << "Service Charges: Rs. " << service << endl;
    cout << "GST: Rs. " << gst << endl;
    cout << "Discount: Rs. " << discount << endl;
    if(total > 5000)
    {
       cout << "Free Delivery Added\n";
    }
    cout << "-----------------------------------\n";
    cout << "Total Payable Amount: Rs. " << total << endl;
    cout << "Enjoy Your Meal \n";
    cout << "=====================================\n";
}
void customerDetails()
{
    cout << "\n===== CUSTOMER DETAILS =====\n";
```

```
    cout << "Customer Name: " << customerName << endl;
    cout << "Contact Number: " << contactNumber << endl;
    cout << "Order Type: " << orderType << endl;
    cout << "Number of Persons: " << persons << endl;
}

int main()
{
    int option;
    registerCustomer();
    do
    {
        cout << "\n===== MAIN MENU =====\n";
        cout << "1. View Food Menu\n";
        cout << "2. Place Order\n";
        cout << "3. Calculate Bill\n";
        cout << "4. View Customer Details\n";
        cout << "5. Exit\n";
        cout << "Enter Choice: ";
        cin >> option;
        if(option == 1)
        {
            displayMenu();
        }
        else if(option == 2)
        {
```

```
            placeOrder();
        }
        else if(option == 3)
        {
            finalBill();
        }
        else if(option == 4)
        {
            customerDetails();
        }
        else if(option == 5)
        {
            cout << "Program Ended\n";
        }
        else
        {
            cout << "Invalid Choice\n";
        }

    } while(option != 5);

    return 0;
}
```

Program 3: Grocery Mart Billing System

Code:

```
#include <iostream>
#include <string>
using namespace std;
// Global Variables
string customerName;
string customerID;
int customerType;
int paymentMethod;
// Global Arrays
string groceryItems[8] = {"Rice 1 KG","Sugar 1 KG","Cooking Oil 1 Litre","Milk Pack","Tea Pack","Flour 5 KG","Eggs Dozen","detergent"};
double prices[8] = {350, 180, 580, 220, 450, 950, 320, 600};
// Cart Array to track quantities of each item ordered
int cartQuantities[8] = {0};

double grossBill = 0;
int currentPoints = 0;
int existingPoints = 0;
int updatedPoints = 0;
// Function to Register Customer
void registerCustomer()
{
   cout << "Enter Customer Name: ";
   cin.ignore();
```

```
    getline(cin, customerName);
    cout << "Enter Customer ID: ";
    getline(cin, customerID);
    cout << "Select Customer Type (1 for Regular, 2 for Member): ";
    cin >> customerType;
    cout << "Select Payment Method (1 for Cash, 2 for Card): ";
    cin >> paymentMethod;
}
// Function to Display Grocery List
void displayGroceryList()
{
    cout << "\n========== GROCERY LIST ==========\n";
    for(int i = 0; i < 8; i++)
    {
        cout << i + 1 << ". " << groceryItems[i]
             << " - Rs. " << prices[i] << endl;
    }
    cout << "==================================\n";
}
// Function to Add Items to Cart
void addItemsToCart()
{
    int itemNo, quantity;
    char choice;
    do
    {
```

```
        displayGroceryList();
        cout << "\nEnter Item Number (1-8): ";
        cin >> itemNo;
        if(itemNo >= 1 && itemNo <= 8)
        {
            cout << "Enter Quantity: ";
            cin >> quantity;

            if(quantity > 0)
            {
                cartQuantities[itemNo - 1] = cartQuantities[itemNo - 1] +
quantity;
                cout << "Added to cart.\n";
            }
            else
            {
                cout << "Invalid Quantity.\n";
            }
        }
        else
        {
            cout << " Invalid Item Number.\n";
        }
        cout << "Add More Items? (y/n): ";
        cin >> choice;
    }
    while(choice == 'y' || choice == 'Y');
```

```
}
// Function to Calculate Gross Bill
double calculateGrossBill()
{
   double total = 0;
   for(int i = 0; i < 8; i++)
   {
      total = total + (prices[i] * cartQuantities[i]);
   }
   return total;
}
// Function to Calculate Sales Tax
double calculateSalesTax()
{
   double tax = 0;
   for(int i = 0; i < 8; i++)
   {
      double itemCost = prices[i] * cartQuantities[i];
      if(i == 7)
      {
         tax = tax + (itemCost * 0.10);
      }
      else
      {
         tax = tax + (itemCost * 0.05);
      }
```

```
    }
    return tax;
}
// Function to Calculate Membership Discount
double calculateMembershipDiscount()
{
    if(customerType == 2)
    {
        return grossBill * 0.07;
    }
    return 0;
}
// Function to Calculate Bill Amount Discount
double calculateBillDiscount(double amountAfterMembership)
{
    if(amountAfterMembership >= 5000 && amountAfterMembership <=
10000)
    {
        return amountAfterMembership * 0.05;
    }
    else if(amountAfterMembership > 10000)
    {
        return amountAfterMembership * 0.10;
    }
    return 0;
}
```

```
// Function to Calculate Card Charges
double calculateCardCharges(double currentTotal)
{
   if(paymentMethod == 2)
   {
      return currentTotal * 0.02;
   }
   return 0;
}
// Function to Calculate Loyalty Points
int calculateLoyaltyPoints(double totalBillAmount)
{
   return totalBillAmount / 100;
}
// Function to Display and Process Final Bill
void displayFinalBill()
{
   grossBill = calculateGrossBill();
   double salesTax = calculateSalesTax();
   double amountWithTax = grossBill + salesTax;
   double memberDisc = calculateMembershipDiscount();
   double amountAfterMember = amountWithTax - memberDisc;
   double billDisc = calculateBillDiscount(amountAfterMember);
   double runningTotal = amountAfterMember - billDisc;
   double cardCharges = calculateCardCharges(runningTotal);
   double finalAmountBeforeLoyalty = runningTotal + cardCharges;
```

```
    // Loyalty Points Logic
    currentPoints = calculateLoyaltyPoints(finalAmountBeforeLoyalty);
    cout << "\n========== GROCERY MART BILL ==========\n\n";
    cout << "Customer Name: " << customerName << endl;
    if(customerType == 2) cout << "Customer Type: Member\n";
    else cout << "Customer Type: Regular\n";
    if(paymentMethod == 2) cout << "Payment Method: Card\n\n";
    else cout << "Payment Method: Cash\n\n";
    cout << "Gross Bill: Rs. " << grossBill << endl;
    cout << "Sales Tax: Rs. " << salesTax << endl;
    cout << "Membership Discount: Rs. " << memberDisc << endl;
    cout << "Bill Discount: Rs. " << billDisc << endl;
    cout << "Card Charges: Rs. " << cardCharges << endl;
    cout << "Generated Loyalty Points: " << currentPoints << endl;
    cout << "Enter Your Existing Loyalty Points (Enter 0 if you're a new
user): ";
    cin >> existingPoints;
    updatedPoints = currentPoints + existingPoints;
    cout << "Loyalty Points after Purchase: " << updatedPoints << endl;
    int redeemChoice;
    cout << "Press 1 to redeem your loyalty points, Press 2 to continue: ";
    cin >> redeemChoice;
    double totalPayable = finalAmountBeforeLoyalty;
    if(redeemChoice == 1)
    {
        totalPayable = totalPayable - updatedPoints;
        if(totalPayable < 0) totalPayable = 0; // Negative bill control
```

```
    }
    cout << "\n-------------------------------------\n";
    cout << "Total Payable Amount: Rs. " << totalPayable << endl;
    cout << "Thank You For Shopping :)\n";
    cout << "=======================================\n";
}
// Function to Display Customer Details
void displayCustomerDetails()
{
    cout << "\n===== CUSTOMER DETAILS =====\n";
    cout << "Customer ID: " << customerID << endl;
    cout << "Customer Name: " << customerName << endl;
    if(customerType == 2) cout << "Customer Type: Member Customer\n";
    else cout << "Customer Type: Regular Customer\n";
    if(paymentMethod == 2) cout << "Payment Method: Card\n";
    else cout << "Payment Method: Cash\n";
    cout << "============================\n";
}
// Main Function at the bottom
int main()
{
    int option;
    registerCustomer();
    do
    {
        cout << "\n===== MAIN MENU =====\n";
```

```
cout << "1. View Grocery Items\n";
cout << "2. Add Items to Cart\n";
cout << "3. Calculate Bill\n";
cout << "4. View Customer Details\n";
cout << "5. Exit\n";
cout << "Enter Choice: ";
cin >> option;
if(option == 1)
{
   displayGroceryList();
}
else if(option == 2)
{
   addItemsToCart();
}
else if(option == 3)
{
   displayFinalBill();
}
else if(option == 4)
{
   displayCustomerDetails();
}
else if(option == 5)
{
   cout << "Program Ended\n";
```

```
        }
        else
        {
            cout << "Invalid Choice\n";
        }
    } while(option != 5);
    return 0;
}
```

Program 4: Online Shopping System

Code:

```
#include <iostream>
#include <string>
using namespace std;
// Global Variables
string userName;
string email;
string city;
int customerType;
int paymentMethod;
// Global Arrays
string products[8] = {"T-
Shirt","Jeans","Shoes","Watch","Handbag","Headphones","Mobile
Cover","Perfume"};
double prices[8] = {1200, 3500, 5000, 2500, 4200, 3000, 700, 2800};
// Cart Array to track quantities
int cartQuantities[8] = {0};
```

```
double productTotal = 0;
// Function to Register User
void registerUser()
{
   cout << "Enter User Name: ";
   cin.ignore();
   getline(cin, userName);
   cout << "Enter Email: ";
   getline(cin, email);
   cout << "Enter City: ";
   getline(cin, city);
   cout << "Select Customer Type (1 for New Customer, 2 for Returning
Customer): ";
   cin >> customerType;
}
// Function to Display Products
void displayProducts()
{
   cout << "\n========== PRODUCT LIST ==========\n";
   for(int i = 0; i < 8; i++)
   {
      cout << i + 1 << ". " << products[i]
           << " - Rs. " << prices[i] << endl;
   }
   cout << "==================================\n";
}
```

```
// Function to Add Products to Cart
void addProductsToCart()
{
   int productNo, quantity;
   char choice;
   do
   {
      displayProducts();
      cout << "\nEnter Product Number (1-8): ";
      cin >> productNo;
      if(productNo >= 1 && productNo <= 8)
      {
         cout << "Enter Quantity: ";
         cin >> quantity;
         if(quantity > 0)
         {
            cartQuantities[productNo - 1] = cartQuantities[productNo - 1] +
quantity;
            cout << "[?] Added to cart.\n";
         }
         else
         {
            cout << "[!] Invalid Quantity.\n";
         }
      }
      else
      {
```

```
            cout << "[!] Invalid Product Number.\n";
        }

        cout << "Add More Items? (y/n): ";
        cin >> choice;

    } while(choice == 'y' || choice == 'Y');
}
//  Function to Calculate Product Total
double calculateProductTotal()
{
    double total = 0;
    for(int i = 0; i < 8; i++)
    {
        total = total + (prices[i] * cartQuantities[i]);
    }
    return total;
}
// Function to Calculate GST (17%)
double calculateGST()
{
    return productTotal * 0.17;
}
// Function to Calculate Delivery Charges
double calculateDeliveryCharges()
{
```

```
    if(city == "Lahore" || city == "lahore" ||
       city == "Karachi" || city == "karachi" ||
       city == "Islamabad" || city == "islamabad")
    {
        return 250;
    }
    else
    {
        return 500;
    }
}

// Function to Calculate Customer Discount
double calculateCustomerDiscount()
{
    if(customerType == 1)
    {
        return productTotal * 0.05;
    }
    else if(customerType == 2)
    {
        return productTotal * 0.10;
    }
    return 0;
}
```

```
// Function to Calculate Order Value Discount
double calculateOrderValueDiscount()
{
   if(productTotal >= 5000 && productTotal <= 10000)
   {
      return productTotal * 0.05;
   }
   else if(productTotal > 10000)
   {
      return productTotal * 0.12;
   }
   return 0;
}
// Function to Calculate Payment Charges (Card = 2.5%)
double calculatePaymentCharges(double currentTotal)
{
   if(paymentMethod == 2)
   {
      return currentTotal * 0.025;
   }
   return 0;
}
// Function to Display Checkout Bill
void displayCheckoutBill()
{
   productTotal = calculateProductTotal();
```

```
double gst = calculateGST();
double delivery = calculateDeliveryCharges();
double customerDisc = calculateCustomerDiscount();
double orderDisc = calculateOrderValueDiscount();
// Total calculation before card processing fee
double intermediateTotal = productTotal + gst + delivery -
customerDisc - orderDisc;
// Ask payment method during checkout
cout << "\nSelect Payment Method:\n";
cout << "1. Cash on Delivery (COD)\n";
cout << "2. Debit/Credit Card\n";
cout << "Enter Choice: ";
cin>> paymentMethod;
double paymentCharges =
calculatePaymentCharges(intermediateTotal);
double finalPayable = intermediateTotal + paymentCharges;
cout << "\n========== ONLINE SHOPPING BILL ==========\n\n";
cout << "User Name: " << userName << endl;
cout << "City: " << city << endl;
if(customerType == 1) cout << "Customer Type: New Customer\n";
else cout << "Customer Type: Returning Customer\n";
cout << "\nProduct Total: Rs. " << productTotal << endl;
cout << "GST: Rs. " << gst << endl;
cout << "Delivery Charges: Rs. " << delivery << endl;
cout << "Customer Discount: Rs. " << customerDisc << endl;
cout << "Order Discount: Rs. " << orderDisc << endl;
cout << "Payment Charges: Rs. " << paymentCharges << endl;
```

```
    cout << "----------------------------------------\n";
    cout << "Final Payable Amount: Rs. " << finalPayable << endl;
    cout << "Thank You for Shopping :)\n";
    cout << "========================================\n";
}
// Function to view user profile details
void viewUserDetails()
{
    cout << "\n===== USER DETAILS =====\n";
    cout << "User Name: " << userName << endl;
    cout << "Email: " << email << endl;
    cout << "City: " << city << endl;
    if(customerType == 1) cout << "Customer Type: New Customer\n";
    else cout << "Customer Type: Returning Customer\n";
    cout << "========================\n";
}
// Main Function
int main()
{
    int option;
    registerUser();
    do
    {
        cout << "\n===== MAIN MENU =====\n";
        cout << "1. View Products\n";
        cout << "2. Add Product to Cart\n";
```

```
cout << "3. Calculate Checkout Bill\n";
cout << "4. View User Details\n";
cout << "5. Exit\n";
cout << "Enter Choice: ";
cin >> option;
if(option == 1)
{
   displayProducts();
}
else if(option == 2)
{
   addProductsToCart();
}
else if(option == 3)
{
   displayCheckoutBill();
}
else if(option == 4)
{
   viewUserDetails();
}
else if(option == 5)
{
   cout << "Program Ended\n";
}
else
```

```
        {
            cout << "Invalid Choice\n";
        }
    } while(option != 5);

    return 0;
}
```

Program 5: Social Media Management System

Code:

```
#include <iostream>
#include <string>
using namespace std;

// Global Variables
string clientName;
string businessName;
int businessType;
int campaignDuration;
int selectedPlatformNo = 0;
int staticPosts = 0;
int reelPosts = 0;
int carouselPosts = 0;
double adBudget = 0;
// Global Arrays
string platforms[3] = {"Instagram", "Facebook", "LinkedIn"};
```

```
double baseCharges[3] = {15000, 12000, 20000};
// Function to Register Client
void registerClient()
{
   cout <<"Enter Client Name: ";
   getline(cin, clientName);
   cout << "Enter Business Name: ";
   getline(cin, businessName);
   cout << "Select Business Type:\n";
   cout << "  1. Small Business\n";
   cout << "  2. Medium Business\n";
   cout << "  3. Corporate Business\n";
   cout << "Enter Choice (1-3): ";
   cin >> businessType;
   cout << "Enter Campaign Duration (in days): ";
   cin >> campaignDuration;
}
// Function to Display Social Media Platforms
void displayPlatforms()
{
   cout << "\n========== AVAILABLE PLATFORMS ==========\n";
   for(int i = 0; i < 3; i++)
   {
      cout << i + 1 << ". " << platforms[i]
           << " (Base Management Charges: Rs. " << baseCharges[i] << ")\n";
   }
```

```
    cout << "=========================================\n";
}
// Function to Select Platform
void selectPlatform()
{
    displayPlatforms();
    cout << "\nEnter Platform Number (1-3) for Campaign: ";
    cin >> selectedPlatformNo;

    if(selectedPlatformNo >= 1 && selectedPlatformNo <= 3)
    {
        cout << "\n--- Enter Campaign Content Requirements ---\n";
        cout << "Number of Static Posts (Rs. 1,000 each): ";
        cin >> staticPosts;
        cout << "Number of Reel/Video Posts (Rs. 2,500 each): ";
        cin >> reelPosts;
        cout << "Number of Carousel Posts (Rs. 1,800 each): ";
        cin >> carouselPosts;
        cout << "Enter Advertisement Budget (Rs.): ";
        cin >> adBudget;
        cout << "Campaign details updated successfully!\n";
    }
    else
    {
        cout << "Invalid Platform Selected. Try again.\n";
        selectedPlatformNo = 0; // Reset if invalid
```

```
    }
}
// Function to Calculate Post Design Cost
double calculatePostDesignCost()
{
    double cost = (staticPosts * 1000) + (reelPosts * 2500) + (carouselPosts
* 1800);
    return cost;
}
// Function to Calculate Ad Handling Fee
double calculateAdHandlingFee()
{
    if(adBudget < 50000)
    {
        return adBudget * 0.05;
    }
    else if(adBudget >= 50000 && adBudget <= 100000)
    {
        return adBudget * 0.08;
    }
    else if(adBudget > 100000)
    {
        return adBudget * 0.10;
    }
    return 0;
}
```

```
// Function to Calculate Extra Duration Charges
double calculateExtraDurationCharges()
{
   if(campaignDuration > 30)
   {
      return (campaignDuration - 30) * 500;
   }
   return 0;
}
// Function to Calculate GST
double calculateGST(double totalServiceCost)
{
   return totalServiceCost * 0.16;
}
// Function to Calculate Discount based on Business Type
double calculateDiscount(double platformAndDesignCost)
{
   if(businessType == 1)
   {
      return platformAndDesignCost * 0.05;
   }
   else if(businessType == 2)
   {
      return platformAndDesignCost * 0.08;
   }
   else if(businessType == 3)
```

```
    {
        return platformAndDesignCost * 0.10;
    }
    return 0;
}
// Function to Display Final Campaign Bill
void displayFinalCampaignBill()
{
    if(selectedPlatformNo == 0)
    {
        cout << "\n Please select a platform and campaign details (Option 2) first.\n";
        return;
    }
    double platformCost = baseCharges[selectedPlatformNo - 1];
    double designCost = calculatePostDesignCost();
    double handlingFee = calculateAdHandlingFee();
    double extraDurationCost = calculateExtraDurationCharges();
    // Total Service Cost subject to GST and Discounts
    double serviceCostBeforeTax = platformCost + designCost + handlingFee + extraDurationCost;
    double gst = calculateGST(serviceCostBeforeTax);
    // Discount usually applies to agency's internal service fees
    double discount = calculateDiscount(platformCost + designCost);
    // Final Calculation: (Service Fees + GST + Ad Budget) - Discount
    double finalCampaignCost = serviceCostBeforeTax + gst + adBudget - discount;
```

```
    cout << "\n========== SOCIAL MEDIA CAMPAIGN BILL
==========\n\n";

    cout << "Client Name: " << clientName << endl;

    cout << "Business Name: " << businessName << endl;

    if(businessType == 1) cout << "Business Type: Small Business\n";

    else if(businessType == 2) cout << "Business Type: Medium
Business\n";

    else if(businessType == 3) cout << "Business Type: Corporate
Business\n";

    cout << "Selected Platform: " << platforms[selectedPlatformNo - 1] <<
endl;

    cout << "Campaign Duration: " << campaignDuration << " Days" << end

    cout << "\nPlatform Management Charges: Rs. " << platformCost <<
endl;

    cout << "Post Design Cost: Rs. " << designCost << endl;

    cout << "Ad Budget: Rs. " << adBudget << endl;

    cout << "Ad Handling Fee: Rs. " << handlingFee << endl;

    cout << "Extra Duration Charges: Rs. " << extraDurationCost << endl;

    cout << "GST: Rs. " << gst << endl;

    cout << "Discount: Rs. " << discount << endl;

    cout << "-----------------------------------------\n";

    cout << "Final Campaign Cost: Rs. " << finalCampaignCost << endl;

    cout <<
"=========================================\n";

}

// Function to Display Client Details

void displayClientDetails()

{
```

```
    cout << "\n===== CLIENT PROFILE DETAILS =====\n";
    cout << "Client Name: " << clientName << endl;
    cout << "Business Name: " << businessName << endl;
    if(businessType == 1) cout << "Business Type: Small Business\n";
    else if(businessType == 2) cout << "Business Type: Medium
Business\n";
    else if(businessType == 3) cout << "Business Type: Corporate
Business\n";
    cout << "Campaign Duration: " << campaignDuration << " Days\n";
    if(selectedPlatformNo != 0)
    {
        cout << "Active Campaign Platform: " <<
platforms[selectedPlatformNo - 1] << endl;
    }
    else
    {
        cout << "Active Campaign Platform: None Selected Yet\n";
    }
    cout << "================================\n";
}
// Main Function
int main()
{
    int option;
    registerClient();
    do
    {
```

```
cout << "\n===== MAIN MENU =====\n";
cout << "1. View Platforms\n";
cout << "2. Select Campaign Platform\n";
cout << "3. Calculate Campaign Cost\n";
cout << "4. View Client Details\n";
cout << "5. Exit\n";
cout << "Enter Choice: ";
cin >> option;
if(option == 1)
{
   displayPlatforms();
}
else if(option == 2)
{
   selectPlatform();
}
else if(option == 3)
{
   displayFinalCampaignBill();
}
else if(option == 4)
{
   displayClientDetails();
}
else if(option == 5)
{
```

```
            cout << "Program Ended\n";
        }
        else
        {
            cout << "Invalid Choice\n";
        }
    } while(option != 5);
    return 0;
}
```

Program 6: Personal Student Diary

Date: 6 - 6 - 2026

Program : Personal Student Diary

```
#include <iostream>
#include <string>
using namespace std;

string regUser = "", regPass = "";
bool isReg = false, loggedIn = false;

string titles[20];
string contents[20];
int totalEntries = 0;

void handleAuth() {
    int choice;
    while (!loggedIn) {
cout << "\n1. Register\n2. Login\n3. Exit\nchoice: ";
cin >> choice;
if (choice == 1) {
  cout << "Username: "; cin >> regUser;
  cout << "Password: "; cin >> regPass;
  isReg = true; cout << "Registered!\n";
  }
```

Date 6-6-2026

```
else if (choice == 2) {
  string u, p;
cout << "Username: "; cin >> u;
cout << "Password: "; cin >> p;
if (isReg && u == regUser && p == regPass) {
    loggedIn = true; cout << "Logged In!\n";
  } else cout << "Invalid Credentials!\n";
  } else exit(0);
}
   Void createEntry() {
if (totalEntries >= 20) { cout << "Diary full!\n"; return; }
cin.ignore();
cout << "Enter Title: "; getline(cin, title[totalEntries]);
cout << "Enter Content: "; getline(cin, contents[totalEntries]);
if(titles[totalEntries].empty() || contents[totalEntries].
empty()) {

cout << "Empty fields not allowed!\n";
} else {

  totalEntries++; cout << "Saved!\n";
  }

}
```

Date: _____

```
Void viewAll() {
cout << "\n---- ALL ENTRIES ----\n";
if (totalEntries == 0) cout << "No Entries found.\n";
for (int i = 0; i < totalEntries; i++)
cout << i + 1 << ". " << titles[i] << endl;
}
  Void readEntry() {
  viewAll(); if (totalEntries == 0) return;
int idx; cout << "Enter Number: "; cin >> idx;
if (idx >= 1 && idx <= totalEntries) {
cout << "\nTitle: " << titles[idx-1] << "\nContent: " <<
contents[idx-1] << endl;
} else cout << "Invalid Number!\n";

}
  Void editEntry() {
  ViewAll(); if (totalEntries == 0) return;
int idx; cout << "Enter Number to edit: "; cin >> idx;
if (idx >= 1 && idx <= totalEntries) {
   cin.ignore();
cout << "New Title: "; getline(cin, titles[idx-1]);
cout << "New content: "; getline(cin, contents[idx-1]);
cout << "Updated!\n";
} else cout << "Invalid Number!\n";
}
```

```
void deleteOne() {
  viewAll(); if (totalEntries == 0) return;
int idx; cout << "Enter Number to Delete: "; cin >> idx;
if (idx >= 1 && idx <= totalEntries) {
  for (int i = idx - 1; i < totalEntries - 1; i++) {
    titles[i] = titles[i+1];
    contents[i] = contents[i+1];
}
   totalEntries--; cout << "Deleted!\n";
} else cout << "Invalid Number!\n";
}

   void searchEntry() {
      cin.ignore();
string keyword; cout << "Enter Keyword to search: "; getline
(cin, keyword);
for (int i = 0; i < totalEntries; i++) {
   if (titles[i].find(keyword) != string::npos){
" << i+1 << "]: " << titles[i] << endl;
        }
     }
  }
```

Date: 6 - 6 - 2026

```
int main() {
  handleAuth();
  nt choice;
do {
  cout<< "\n=== STUDENT DIARY( "<< totalEntries << "/20===\n";
cout<< "1. Create Entry\n2. View All\n3. Read Entry\n4. Edit Entry\n5
Delete one\n6. Delete All\n7. Search\n8. Exit\nChoice: ";
cin>> choice;
if(choice == 1) createEntry();
else if (choice == 2) viewAll();
else if(choice == 3) readEntry();
else if( choice == 4) editEntry();
else if( choice == 5) deleteOne();
else if( choice == 6){ totalEntries=0; cout<< "cleared!\n"; }
else if (choice == 7) SearchEntry();
} while (choice != 8);

  return 0;

}
```

PROJECT REPORT:
PERSONAL STUDENT DIARY

1. Project Introduction:

This project is a console-based Personal Student Diary application developed in C++. It allows students to securely save, view, read, edit and delete their daily text entires. The System works completely in memory (RAM) using 1D arrays, which means data is temporary and resets when the program close.

2. Main Features:

- Authentication System: Users must register a username and password, then Login before accessing the diary dashboard.
- CRUD Operations: Supports Creating, Reading, Updating (Editing), and Deleting diary entires.
- Input Validation: Prevents errors if the user enters wrong entry numbers or leaves field empty.

Date: 6-6-2026

- Search function (Bonus): Allows users to search any specifics entry by trying a keyword related to the title.

3. Program [redacted] Architecture And Data Storage:

The Application uses two parallel 1D global arrays to store details up to a maximum of 20 entires:

1 titles[20] (To store the heading of the entry)

2 contents[20] (To store the description or short notes)

A global variable totalEntires keeps track of the current number of active records. When an entry is deleted items to keep the array clean and continuous.

4. Function Breakdown:

- handleAuth(): Manages user account registration and secure login sequence.

- createEntry(): Input new titles and contents after checking the size limit.

Date: 6-6-2026

- viewAll(): Loops through the arrays to display of a specific entry selected by the user.
- readEntry(): Displays full text details of a specific entry selected by the user.
- editEntry(): Overwrites existing values inside the array at a chosen position.
- deleteOne(): Shuts the gap by shifting elements to remove on specific item.
- SearchEntry(): Used .find() string method to fetch titles matching a keyword.